Case ID	Student Name	LLM Model	Original Code Snippet
1	Student Name	Claude Sonnet 4.6	train_test_split(X, y, test_size=0.2, ...)
2	Student Name	Claude Sonnet 4.6	X_test_scaled = scaler.transform(X_test)
3	Student Name	Claude Sonnet 4.6	model.fit(X_train_s, y_train)
4	Student Name	Claude Sonnet 4.6	RandomForestClassifier(n_estimators=100, ...)
5	Student Name	Claude Sonnet 4.6	model.fit(X_train_s, y_train) y_pred = model.predict(X_test_s)

6	Student Name	Claude Sonnet 4.6	y_pred = model.predict(X_test_s)
7	Student Name	Claude Sonnet 4.6	train_test_split(..., random_state=42) RandomForestClassifier(..., random_state=42)
8	Student Name	Claude Sonnet 4.6	train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
9	Student Name	Claude Sonnet 4.6	RandomForestClassifier(n_estimators=100, max_depth=5, ...)
10	Student Name	Claude Sonnet 4.6	accuracy_score(y_test, y_pred) classification_report(y_test, y_pred)

New Code Snippet (with bug)	Initial Self-Reflection
train_test_split(X, y, test_size=1.5, ...)	I think there is an error because I used 1.5 as the test_size. The number seems too large. Maybe it should be between 0 and 1.
X_test_scaled = scaler.fit_transform(X_test)	I changed transform to fit_transform on the test set. The code still runs and accuracy went up slightly, so maybe it doesn't matter which one I use.
model.fit(X_train_s, y)	X_train_s has 400 rows (80% of 500) but y still has 500 rows. So I'm passing the full labels y instead of y_train which only has 400 rows.
RandomForestClassifier(n_estimators="100", ...)	The error is because '100' is a string but n_estimators needs an integer.
y_pred = model.predict(X_test_s) # forgot model.fit()	The model is not trained yet. I need to call fit before predict. I accidentally deleted the fit line.

y_pred = model.predict(X_test_s[:, :5])	I sliced X_test_s to only 5 columns but the model was trained on 13 features. Sklearn validates feature count at prediction time.
train_test_split(...) RandomForestClassifier(...)	I removed random_state so different random numbers are used each run. Since the dataset is the same I think the variance should be small and acceptable.
train_test_split(X, y, test_size=0.2, random_state=42, stratify=y_train)	y_train has only 400 samples but X and y have 500, so stratify must match the full data length.
RandomForestClassifier(n_estimators=100, max_depth=0, ...)	max_depth=0 means no splits at all, which is meaningless, so sklearn rejects it with a minimum of 1.
accuracy_score(y_pred, y_test) classification_report(y_pred, y_test)	I swapped the arguments. For accuracy_score it doesn't matter because accuracy is symmetric. I think the swap is fine since the numbers are the same.

GenAI Label	GenAI Guidance
Bad	Your reflection notices that 1.5 is 'too large' but only scratches the surface. Consider: what does test_size actually represent? Is it always a fraction? What would an integer value like 2 or 150 mean in this context? Think carefully about the two distinct ways train_test_split interprets this parameter depending on whether the value is a float or an integer — and why 1.5 fits neither category.
Bad	You noticed the change but concluded it 'doesn't matter' because the shapes are the same — this is the crux of the problem. Ask yourself: what does fit_transform do differently from transform? Why does a StandardScaler need to be 'fitted' at all? If you fit the scaler separately on the test set, what statistics is it now using to scale those values — and should those statistics come from the test set? Think about what information the test set is 'leaking' into your evaluation pipeline.
Good	Your reflection correctly identifies the root cause: X_train_s has 400 rows (80% of 500) but y still has 500 rows. You've identified the split mismatch and understand why it happens. Proceed with your own reasoning to fix the issue.
Good	Your reflection correctly identifies that the type mismatch is the issue — '100' is a string while n_estimators requires an integer. You understand the Python distinction between numeric types and string types. Proceed with your own fix.
Good	You've correctly identified that the model was never trained before predict was called. You understand the required ordering of fit → predict in sklearn's API. Proceed to add the missing fit call.

Good	Your reflection correctly identifies the issue: the model was trained on 13 features but prediction was attempted with only 5. You also correctly note that sklearn stores and validates feature count at fit time. Proceed with your fix.
Bad	You correctly note that randomness is reintroduced, but then dismiss it as acceptable. Think more carefully: in a research or production context, why does reproducibility matter? If you ran this code today and got accuracy 0.85, then again tomorrow and got 0.91, which one would you report? How does removing random_state affect your ability to debug, compare experiments, or share results with others? Is small variance the only concern — or is the principle of reproducibility itself important?
Good	Your reflection correctly identifies the mismatch: stratify=y_train passes a 400-length array to a split operating on 500-row data. You understand that the stratify argument must match the length of the arrays being split. Proceed with your fix.
Good	Your reasoning is sound: a tree with max_depth=0 could make no splits and thus could not learn anything, so sklearn enforces a minimum of 1. You also correctly identified the valid range [1, inf) or None. Proceed with your fix.
Bad	You concluded the swap is fine because accuracy 'is the same number.' But consider: what exactly is the function signature of accuracy_score(y_true, y_pred)? Why does the function distinguish between y_true and y_pred at all if the result is always symmetric? Now look at classification_report — what does it label as 'precision' and 'recall' for each class? If you swap which array is 'true' and which is 'predicted', does the semantic meaning of precision and recall change even if the matrix looks similar? What could go wrong if you used this flipped report to make decisions about your model?

New Self-Reflection
I now understand that test_size accepts either a float in (0.0, 1.0) representing a proportion, or an integer ≥1 representing an absolute count of samples. 1.5 is neither a valid proportion nor a valid integer, so sklearn rejects it. I should use test_size=0.2 (20%) or test_size=100 (100 samples).
fit_transform re-learns mean and std from the test set, so each feature is scaled using test-set statistics rather than training-set statistics. This is data leakage: the model evaluation uses information that 'leaked' from the test set into preprocessing. This inflates metrics artificially and hides the true generalization gap.
N/A — reflection was labeled Good, no revision needed.
N/A — reflection was labeled Good, no revision needed.
N/A — reflection was labeled Good, no revision needed.

N/A — reflection was labeled Good, no revision needed.
Reproducibility is a fundamental requirement for scientific validity. Without a fixed random_state, both the train/test split and the tree-building are random, so two colleagues running the same code get different accuracies. This makes debugging impossible, comparisons unfair, and results unpublishable. Even if variance is empirically small, the principle matters.
N/A — reflection was labeled Good, no revision needed.
N/A — reflection was labeled Good, no revision needed.
Accuracy happens to be symmetric but classification_report is not. When y_pred is treated as y_true, the 'precision' and 'recall' labels become semantically incorrect — precision is computed as if the model's guesses were the ground truth. This would lead to wrong conclusions about which class the model struggles with most. The API contract matters even when the output looks similar.